Lesson 1. Introduction to NumPy

Exercise 1: Basic Array initialization

Use the *np.array()* function to create a one-dimensional array (ndarray) containing integers from 1 to 5. Print the array to the screen.

Exercise 2: Creating Multidimensional Arrays

Create a 2-dimensional array with a shape of 2 x 3 (2 rows, 3 columns) containing any numerical values. Use the *.shape* attribute to verify the dimensions of the created array.

Exercise 3: Specifying Data Types (dtype)

Create an array from the list [1, 2, 3] but explicitly set the data type of the elements to complex numbers (*complex*). Observe the format of the elements in the output.

Exercise 4: Setting Minimum Dimensions

Use the *ndmin* parameter to create an array from the list [10, 20, 30] such that the resulting array has at least 2 dimensions.

Exercise 5: Checking Number of Dimensions (ndim)

Create a 3-dimensional array by combining *np.arange(24)* with the *.reshape(2, 4, 3)* method. Then, use the **.ndim** attribute to confirm the number of dimensions of the array.

Exercise 6: Analyzing Element Size (itemsize)

1. Create one array with the **int8** type and another with the **float32** type.
2. Use the *.itemsize* attribute to display how many bytes each element of each array occupies in memory.

Exercise 7: Basic Type Casting

Create an array containing the values [10, 20, 30]. Use the *dtype* parameter to:

- Cast the array's data type to an 8-bit integer (**int8**).
- Print the array and its *dtype* attribute to the screen.

Exercise 8: Using Character Codes

Instead of using keywords like *np.int32*, NumPy allows the use of character codes to identify data types.

- Create an array with a 32-bit floating-point data type using the character code *'f4'*.
- Create another array containing strings using the code *'S20'* (representing a byte-string with a maximum length of 20).

Exercise 9: Simple Structured Data Type

Use the *np.dtype* function to create a structured data type named **inventory**.

- This data type should consist of a single field: '*item_code*' with an integer type (**int32**) .
- Initialize an *ndarray* applying this **inventory** type with the following item codes: 1001, 1002, 1003.

Exercise 10: Complex Multi-field Structure

Construct a structured data type representing book information with the following fields:

- Field '**title**': byte-string type ('**S30**').
- Field '**pages**': 16-bit integer type ('**i2**').
- Field '**price**': floating-point type ('**f4**'). After defining the type, create an array to store information for at least 2 books.

Exercise 11: Accessing Structured Data

Using the book information array you created in **Exercise 10**: Write a command to access and print only the '**title**' field for all books in the array.

Exercise 12: Resizing via the Shape Attribute

Initialize a 1-dimensional array with 12 elements (e.g., *np.arange(12)*).

- Directly modify the *.shape* attribute of that array to convert it into a 4 x 3 array.
- Print the array after the change to observe how the elements are rearranged.

Exercise 13: Analyzing Array Status via Flags

Create a basic NumPy array and print all the information provided by the *.flags* attribute.

- Based on the descriptions in the tutorial, explain the meaning of the **C_CONTIGUOUS** and **F_CONTIGUOUS** parameters.
- Check the value of **OWNDATA** and explain what it signifies regarding the array's memory management.

Exercise 14: Initializing Arrays with Zeros

Use the *numpy.zeros* function to:

- Create a 1D array of five zeros with the default data type.
- Create another 1D array of five zeros but cast the data type to **int**.
- Print both and observe the difference in the decimal points.

Exercise 15: Creating Arrays of Ones

Use the *numpy.ones* function to:

- Create a 2D array with a 2 x 2 shape.
- Set the data type for this array to **int**.
- Print the result and verify that all elements are 1.

Exercise 16: Uninitialized Array Creation

Use the *numpy.empty* function to create a 3 x 2 array with an integer data type.

- Print the array.
- Explain why the elements show random values instead of zeros or ones.

Exercise 17: Customizing Storage Order

The functions *empty*, *zeros* and *ones* all support the **order** parameter.

- Create a 3 x 3 *zeros* array with **order='F'** (Fortran-style).
- Create a similar array with **order='C'** (C-style).
- Use the .*flags* attribute to confirm the difference in memory layout.

Exercise 18: Initializing with Structured Data Types

Combine knowledge from **Data Types** and **Array Creation**:

- Use *numpy.zeros* to create a 2 x 2 array with a custom dtype consisting of two fields: **'x'** (4-byte integer) and **'y'** (4-byte integer).
- Print the array to see how NumPy represents the pairs of (0,0).

Exercise 19: Converting Lists and Tuples to Arrays

Use the *numpy.asarray* function to:

- Create an array from a list of floats: [1.5, 2.5, 3.5].
- Create another array from a tuple of integers: (10, 20, 30).
- Print the results and verify the **dtype** of the array created from the list.

Exercise 20: Array from a List of Tuples

Use the *numpy.asarray* function to:

- Convert the following list of tuples into an array: [(1, 2), (3, 4)] .
- Observe and print the **shape** of the resulting array.

Exercise 21: Interpreting Buffers as Arrays

Use the *numpy.frombuffer* function to convert a string into an array:

- Initialize the string **s = 'Python'**.
- Create an array using the **dtype='S1'** parameter to read individual characters.
- Explain the utility of this function when working with objects exposing the buffer interface.

Exercise 22: Building Arrays from Iterators

Combine Python's *range()* and *iter()* functions with *numpy.fromiter*:

- Create an iterator object from the sequence **range(10)**.
- Convert this iterator into a NumPy array with a **float** data type.
- Try limiting the items read by using the **count=5** parameter and observe the output.

Exercise 23: Comparing *numpy.array* and *numpy.asarray*

Based on the introduction in the document:

- State the primary difference regarding parameters between *numpy.array* and *numpy.asarray*.
- Write code using *asarray* to convert a list of tuples with inconsistent lengths (e.g., [(1,2,3), (4,5)]) and describe the resulting array's elements.

Exercise 24: Basic Sequences with arange

Use the *numpy.arange* function to:

- Create an array containing integers from 0 to 4.
- Create another array containing numbers from 0 to 4 with a **float** data type.
- Print the results and explain why the stop value is not included in the array.

Exercise 25: Sequences with Custom Steps

Use the *numpy.arange* function with specific parameters to:

- Create an array starting from 10, ending before 20, with a step size of 2.
- Print the resulting array and identify its **shape**.

Exercise 26: Evenly spaced samples with *linspace*

Use the *numpy.linspace* function to:

- Generate an array of 10 evenly spaced samples between 1 and 5.
- Repeat the task but set the **endpoint** parameter to **False** and observe if the value 5 is still included.

Exercise 27: Understanding the **retstep** parameter

The *linspace* function has a specific parameter called **retstep**.

- Create a sequence of 5 numbers from 0 to 10 and set **retstep=True**.
- Print the result and explain the returned data structure. What does the additional value represent ?

Exercise 28: Comparing *arange* vs. *linspace*

Write code to produce the same sequence [10, 12, 14, 16, 18, 20] using two methods:

- Method 1: Use *numpy.arange*.
- Method 2: Use *numpy.linspace*.
- Explain the difference in logic between using the **step** parameter (in arange) versus the **num** parameter (in linspace).

Exercise 29: Basic Element Access

Given the array **arr = np.array([10, 20, 30, 40, 50])**. Write a command to extract the first element and the fourth element of the array based on the *zero based indexing* mechanism.

Exercise 30: Simple Array Slicing

Given the array **array8 = np.array([-2, 2, 6, 10, 14, 18, 22])**. Perform a slicing operation to extract a sub-array containing the values **[10, 14]**.

Exercise 31: Reversing an Array

Given the array **array8 = np.array([-2, 2, 6, 10, 14, 18, 22])**. Write a command using the slicing technique with a step value to completely reverse the order of the elements in the array.

Exercise 32: Extracting Data by Column

Given the array **array9 = np.array([[-7, 0, 10, 20], [-5, 1, 40, 200], [-1, 1, 4, 30]])** with dimensions 3x4. Write a command to extract all elements belonging to the 3rd column of this array.

Exercise 33: Sub-array Slicing

Given the array **array9 = np.array([[-7, 0, 10, 20], [-5, 1, 40, 200], [-1, 1, 4, 30]])** with dimensions 3x4. Perform a slicing command to extract a sub-array consisting of elements from rows 2 and 3 that also belong to columns 1 and 2.

Exercise 34: Flexible use of ":"

Rewrite the command from Exercise 32 (accessing the entire 3rd column) by using the : symbol for rows instead of specifying a concrete index range like 0:3.

Exercise 35: Extracting Matrix Boundaries

Given any 2-D matrix. Write a command to extract all columns except for the first column and the last column. (Hint: Use negative indexing or an end index range).

Exercise 36: Handling Index Errors (IndexError)

Write a code snippet that attempts to access an index exceeding the array's dimensions (e.g., accessing the 5th column when the array only has 3 columns). Observe the returned error and explain why it occurs based on the information in the document.

Exercise 37: Slicing and Assignment

Create a 2-D array *array_test* from the following nested list: *[[1, 2, 3, 4], [5, 6, 7, 8], [9, 10, 11, 12]]*.

1. Extract a sub-array containing the last 2 rows and the last 2 columns.
2. Assign the value 0 to all elements in the last column using slicing.

Exercise 38: Basic Element-wise Operations

Given the following two 2-D arrays: **array1 = np.array([[3, 6], [4, 2]])** and **array2 = np.array([[10, 20], [15, 12]]).** Write commands to perform:

- Element-wise addition of array1 and array2.
- Element-wise subtraction of array1 from array2.

Exercise 39: Exponentiation and Modulo

Given the following two 2-D arrays: **array1 = np.array([[3, 6], [4, 2]])** and **array2 = np.array([[10, 20], [15, 12]]).**

- Calculate the cube (exponentiation to the power of 3) for all elements in **array1**.
- Perform the modulo operation of elements in **array2** by the corresponding elements in **array1**.
- Explain the necessary condition regarding the shape of the two arrays when performing these operations.

Exercise 40: Matrix Multiplication vs. Element-wise Multiplication

Given the following two 2-D arrays: **array1 = np.array([[3, 6], [4, 2]])** and **array2 = np.array([[10, 20], [15, 12]]).**

- Perform element-wise multiplication between the two arrays.
- Perform matrix multiplication between the two arrays using the **@ operator**.
- Observe the results and state the difference between these two operations based on the output.

Exercise 41: Array Transpose

Create a 3x4 array array3 as follows: **array3 = np.array([[10, -7, 0, 20], [-5, 1, 200, 40], [30, 1, -1, 4]])**.

- Perform a transpose on **array3** (turning rows into columns and columns into rows).
- Print the transposed array and state its new shape.
- Check whether the original **array3** changes after the transpose operation.

Exercise 42: Multi-dimensional Sorting

Given the array **array5 = np.array([[10, -7, 0, 20], [-5, 1, 200, 40], [30, 1, -1, 4]])**.

- Perform column-wise sorting on **array5** by using the parameter **axis=0**.
- After sorting column-wise, perform row-wise sorting on that same array using the parameter **axis=1** (or the default setting).
- State whether NumPy performs sorting in ascending or descending order by default.

Exercise 43: Basic Vertical Concatenation

Given two 2-D arrays of the same size as follows: **A = np.array([[1, 2], [3, 4]])** and **B = np.array([[5, 6], [7, 8]])**. Perform the following tasks:

- Use the *np.concatenate()* function to join arrays A and B vertically (default **axis=0**) to create a new 4x2 array.
- Print the resulting array and state the total number of rows in the new array.

Exercise 44: Column-wise Concatenation and Shape Error Handling

Given two arrays with different dimensions: **array1 = np.array([[10, 20], [-30, 40]])** (size 2x2) and **array2 = np.zeros((2, 3), dtype=int)** (size 2x3). Perform the following operations:

- Attempt to execute the command **np.concatenate((array1, array2), axis=0)**. Observe the returned error and explain it based on the rule that "all the input array dimensions except for the concatenation axis must match exactly".
- Successfully concatenate these two arrays horizontally (**axis=1**) to create a 2x5 array.Explain why, in this case, concatenation along **axis=1** is valid while **axis=0** is not.

Exercise 45: Basic Descriptive Statistics

Given a 1-D array **arrayA = np.array([1, 0, 2, 3, 6, 8, 4, 7])**. Write commands to:

- Find the maximum and minimum values of the array.
- Calculate the sum of all elements in the array.
- Calculate the average (mean) value and the standard deviation of the array.

Exercise 46: Axis-based Statistical Operations

Given a 2-D array **arrayB = np.array([[3, 6], [4, 2]])**. Perform the following tasks using the axis parameter:

- Find the maximum value for each row (**axis=1**) and each column (**axis=0**).
- Calculate the sum of elements for each row.
- Find the average value for each column.

Exercise 47: Advanced Statistical Analysis on Matrices

Create a 2-D array representing a small dataset.

- Find the standard deviation of the array for each row and each column.
- Calculate the overall mean of the entire 2-D array.
- Explain the relationship between the **size** of the array and the results of the **sum()** and **mean()** functions (Hint: Mean is the sum divided by the total number of elements).

Exercise 48: Basic Loading and Saving

Assume you have a text file named **data.txt** containing student scores, where values are separated by commas and the first row is a header.

- Write a command using *np.loadtxt()* to load data from this file into a NumPy array named **student_data**. Make sure to skip the header row and specify the data type as integer (**int**).
- After loading, save the entire **student_data** array into a new file named **backup_data.csv** using a comma as the delimiter and the integer format.

Exercise 49: Extracting Columns using Unpack

Use the **data.txt** file which has the column structure: **RollNo**, **Marks1**, **Marks2**, **Marks3**.

- Use the *np.loadtxt()* function with the **unpack=True** parameter to load the columns directly into 4 separate arrays: **roll_no**, **m1**, **m2**, and **m3**.
- Calculate a new array named **total_marks** by adding **m1 + m2 + m3**.
- Save the **total_marks** array into a text file named **FinalResult.txt**.

Exercise 50: Handling Missing Data and Cleaning

You have a data file **dataMissing.txt** containing some missing values or non-numeric data (which results in **nan** when loaded).

- Use the *np.genfromtxt()* function to load this file. Use the **filling_values** parameter to replace all missing or invalid values with the number **-999**.
- Create a new array containing only the rows that do not have any **-999** values (data filtering).
- Save this cleaned data array into a file named **CleanedData.txt** in float format, rounded to 2 decimal places.

Key Functions and Parameters:

- **np.loadtxt()**: Used for standard data files that do not have missing values.
- **np.genfromtxt()**: More flexible than **loadtxt()** as it can handle missing values in the data file.
- **skiprows / skip_header**: Parameters used to skip header rows so they are not loaded into the array.
- **unpack=True**: When set to True, the returned array is transposed, allowing you to extract columns as separate arrays.
- **filling_values**: A parameter in **genfromtxt()** used to convert missing values or character strings into a specific value.
- **np.savetxt()**: The function used to save a NumPy array to a text file.